Informe final, Desafío II, UdeABook

Informática II · Universidad de Antioquia · 2026-2

Oscar Trujillo

Medellín, 18 de octubre de 2026

1. Introducción

Este es el informe final del segundo desafío. El programa quedó terminado: carga desde archivos una red social de 1.200 usuarios, con 20.710 amistades, 900 solicitudes pendientes, 15.000 mensajes y 7.283 fotos repartidas en unos 2.400 álbumes, y sobre esos datos funcionan el inicio de sesión, la administración de amistades y solicitudes, las sugerencias, los mensajes, los álbumes con sus me gusta y las fotos de perfil de la pantalla principal. Todo está hecho con clases y estructuras propias, sin STL ni librerías externas, y después de cada operación el menú imprime la memoria usada y las iteraciones que costó. Como el diseño y sus razones ya quedaron explicados en el pre informe, aquí me concentro en cómo quedó todo al final, en lo que cambió sobre la marcha y, sobre todo, en las mediciones que respaldan las decisiones de eficiencia.

2. Cómo quedó el programa

El diseño del pre informe se sostuvo casi completo. Son doce clases en dos niveles: las de apoyo (Texto, Fecha, ListaDinamica, MatrizBits y Medicion), que no saben nada de la red social, y las del dominio (Mensaje, Foto, Album, Usuario, RedSocial, Archivos y Menu). La regla que ordena todo es tener claro quién es dueño de qué: RedSocial crea y destruye los usuarios y los mensajes; cada usuario tiene sus álbumes por valor y cada álbum sus fotos, porque no se comparten con nadie; y de los mensajes, que sí le sirven a dos personas a la vez, cada usuario guarda solo punteros, así ningún mensaje existe dos veces en memoria. Archivos arma la red leyendo los seis archivos de texto y la guarda al salir, y Menu es la única clase que habla con la consola.

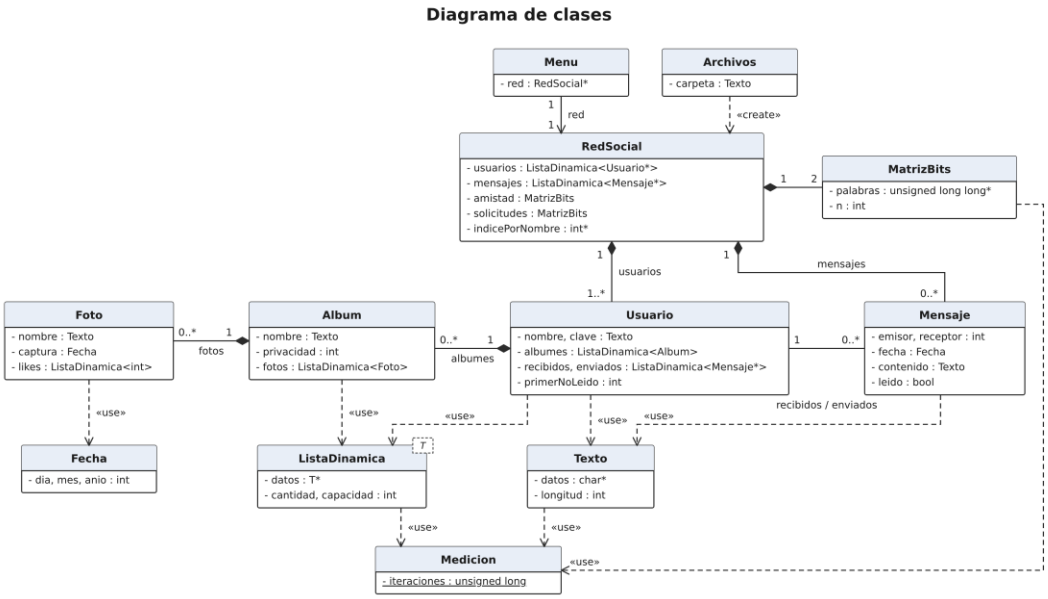


Figura 1. Diagrama de clases (vista resumida; el detallado está en el repositorio).

De lo que pide el enunciado sobre POO: Texto, ListaDinamica y MatrizBits manejan memoria dinámica con constructor de copia, destructor y operador de asignación; Texto sobrecarga ==, < y +, Fecha sobrecarga ==, < y <=, y ambas tienen el operador << como función amiga; ListaDinamica es una plantilla, y Medicion usa miembros estáticos para que cualquier clase sume iteraciones sin andar pasando un contador.

3. Qué cambió respecto al pre informe

Tres cosas, todas al chocar con la práctica:

El índice de nombres se construye al terminar la carga, no en la primera búsqueda. Inicialmente había pensado construir el índice de nombres cuando se realizara la primera búsqueda. Sin embargo, las pruebas

mostraron que el primer inicio de sesión requería unas 83.000 iteraciones adicionales debido al ordenamiento, mientras que los siguientes solo necesitaban 181. Por eso decidí construir el índice al terminar la carga, para que el costo del inicio de sesión fuera más estable.

- Al moverlo a la carga, todo login cuesta lo mismo: 181 iteraciones.
- El programa busca solo la carpeta de datos. Qt Creator no ejecuta desde la carpeta del proyecto sino desde la de compilación, y al principio el programa no encontraba los archivos. En vez de depender de configurar el directorio de trabajo a mano, el main prueba datos, ../datos, ../../datos y así sucesivamente, y guarda en la misma carpeta que encontró.
- Un mensaje no puede llevar una fecha anterior al último mensaje de sus participantes. La búsqueda por rango de fechas depende de que las listas estén ordenadas, y como la fecha la escribe el usuario, alguien podía dañar el orden metiendo un mensaje "viejo". Preferí validarlo al enviar y mantener la garantía, y lo dejo anotado como supuesto.

Además escribí un programa aparte (está en la carpeta generador del repositorio) que produce los seis archivos de prueba con los 1.200 usuarios, cuidando que los mensajes queden en orden cronológico y que solo se escriban entre amigos.

4. Mediciones

Los números salen del contador de Medicion, que suma uno en cada vuelta de cada ciclo de mis estructuras. Para las comparaciones con la alternativa sencilla hice pruebas aparte recorriendo las listas, con los mismos datos. El promedio de la búsqueda por nombre es el de buscar a los 1.200 usuarios uno por uno, para no depender de la suerte de dónde quede el que se busca.

Operación (con 1.200 usuarios y 15.000 mensajes)	Iteraciones	Alternativa sencilla
Cargar los seis archivos (una vez, al abrir)	2.345.682	—
Iniciar sesión (buscar el nombre y validar la clave)	181	—
Buscar un usuario por nombre (promedio de buscar los 1.200)	82	743 (recorriendo)
¿A y B son amigos?	0 (leer un bit)	recorrer su lista
Amigos en común (promedio de 200 parejas)	20	1.200
Contar los mensajes sin leer	0 (una resta)	recorrer recibidos
Conversación entre dos fechas (3 encontrados entre 15.000)	35	15.000
Listar los amigos de un usuario (39)	1.260	—
Sugerencias de amistad (las 10 mejores)	24.703	—
Guardar todo (una vez, al salir)	2.185.592	—

La lectura que hago de la tabla: las operaciones que un usuario repite todo el tiempo quedaron baratas y las dos caras ocurren una sola vez. Buscar por nombre bajó de 743 a 82 iteraciones gracias al índice ordenado con búsqueda binaria. Los amigos en común bajaron de 1.200 a 20 porque el AND de las filas procesa 64 casillas por vuelta y el conteo de unos con *x & (x-1)* solo da una vuelta por cada bit encendido. Contar los no leídos no recorre nada porque los nuevos siempre quedan al final de la lista y basta restar dos números. Y la conversación entre dos fechas no revisa los 15.000 mensajes: una búsqueda binaria encuentra dónde empieza el rango en los recibidos y en los enviados, y de ahí se avanza mezclando las dos listas para que salga en orden.

En memoria, toda la red cargada ocupa 3.282.656 bytes, unos 3,1 MB. La parte que más se ahorró fue la de las relaciones: las dos matrices de bits (amistades y solicitudes) ocupan 364.800 bytes, mientras que las mismas matrices con bool, que ocupa un byte entero, pesarían 2.880.000: casi ocho veces más.

5. Limitaciones y posibles mejoras

- Listar los amigos de alguien recorre los 1.200 bits de su fila uno por uno (1.260 iteraciones). Se podría avanzar palabra por palabra y saltar las que están en cero, pero como es una operación de mostrar en pantalla y su salida puede ser larga de todas formas, no lo prioricé.

- Las sugerencias son la consulta más cara (24.703 iteraciones) porque revisan a todos los no amigos. En un grafo tan disperso como el de prueba (35 amigos en promedio) saldría más barato recorrer directamente a los amigos de mis amigos, aunque tocaría manejar repetidos y de todas maneras calcular los amigos en común de cada candidato para ordenarlas. Queda como la mejora que haría si tuviera que escalar esto.
- El guardado recorre media matriz para escribir las parejas de amigos (es la mayor parte de sus 2,1 millones de iteraciones). Como pasa una sola vez al salir, preferí dejarlo simple.

6. Conclusiones

Al desarrollar este desafío me di cuenta de que elegir bien las estructuras de datos facilita mucho las operaciones del programa. Por ejemplo, usar una matriz de bits para las amistades y un índice ordenado para buscar usuarios permitió reducir el trabajo de varias consultas. Esto lo pude comprobar con las mediciones, especialmente al comparar la búsqueda por nombre con el recorrido de una lista.